# Teleoperación con Sensado de Fuerza Física entre dos xArm Lite 6: Diseño, Implementación y Evaluación Experimental

Gerardo Fregoso Jiménez*, Luis Fernando Ruíz Neria*, Daniel Eduardo Hinojosa Alvarado*, José Raúl Arredondo López*
*Computational Robotics Lab, Tecnológico de Monterrey, Monterrey, N.L., México
{A00838645, A00839528, A00838156, A01742153}@tec.mx

***Resumen*—This paper presents the design, implementation, and experimental evaluation of a bilateral teleoperation system with force sensing between two xArm Lite 6 six-degree-of-freedom manipulators. The system is built on ROS 2 Humble and MoveIt Servo, running at 50 Hz using DDS (rmw_fastrtps) as the inter-process communication layer. The master arm is governed by a Cartesian admittance controller ($\boldsymbol{M}_d\dot{\boldsymbol{v}} + \boldsymbol{B}_d\boldsymbol{v} = \boldsymbol{F}_{op} - \alpha\boldsymbol{F}_{slave}$, with $\boldsymbol{M}_d = \mathbf{diag}(2,2,2)$ kg, $\boldsymbol{B}_d = \mathbf{diag}(10,10,10)$ N·s/m, $\alpha = 0.3$) that converts the net operator force into a Cartesian velocity command sent as `TwistStamped` to MoveIt Servo. The slave arm tracks the master end-effector pose via a feedforward-plus-proportional impedance law ($\boldsymbol{v}_{cmd} = \boldsymbol{v}_{ff} + \boldsymbol{K}_p\boldsymbol{e}$, $\boldsymbol{K}_p = \mathbf{diag}(5,5,5)$ s$^{-1}$) also executing through MoveIt Servo. Cartesian force at the slave end-effector is estimated without an external F/T sensor by computing joint-torque residuals: the damped least-squares pseudo-inverse of the geometric Jacobian ($\lambda = 0.1$) maps gravity-compensated residual torques to a six-component wrench, filtered at $f_c = 5$ Hz. Two physical force sensors, each based on an ESP32 with micro-ROS over WiFi, were designed and built to provide direct operator force input: the *master* sensor (node `force_sensor_hw`, topic `/teleop/operator_force_direct`, $k = 1000$ N/m, $\pm 3$ N joystick) and the *slave* sensor (node `slave_force_sensor_hw`, topic `/teleop/slave_force_direct`, $k = 1500$ N/m, $\pm 6$ N joystick), both publishing `WrenchStamped` at 50 Hz. An Isaac Sim integration is also provided for hardware-free validation, with the simulator acting as the physics backend while MoveIt Servo and the teleop controllers run unmodified on the ROS 2 side. Experiments demonstrate functional teleoperation with joint tracking RMS error $<$ [X]° and force estimation error $<$ [X] N against a precision scale reference.**

***Index Terms*—teleoperación, sensado de fuerza, xArm Lite 6, control de impedancia, retroalimentación háptica, micro-ROS, admitancia**

## I. Introducción

La teleoperación con retroalimentación de fuerza constituye un paradigma fundamental en robótica cuando se requiere que un operador humano interactúe con entornos remotos o peligrosos sin presencia física [7]. El xArm Lite 6, desarrollado por UFACTORY [1], es un manipulador de 6 grados de libertad (DOF) con capacidad de control de torque que lo hace idóneo para experimentos de sensado de fuerza por torque articular.

**(i) Contexto: Motivación**

El reto central de este proyecto es cerrar el lazo de fuerza entre dos brazos robóticos: el robot **Maestro**, operado por el humano, y el robot **Esclavo**, que interactúa con el entorno. La fuerza de contacto estimada en el esclavo debe reflejarse como torque en las articulaciones del maestro, creando la sensación háptica que guía al operador.

### *I-A. Contexto y Motivación*

La teleoperación con sensado de fuerza permite extender la capacidad sensorial del operador humano más allá de sus limitaciones físicas, ya sea en distancia, escala o condiciones ambientales adversas. En el contexto industrial, la manipulación remota de materiales peligrosos (residuos nucleares, explosivos) requiere que el operador perciba las fuerzas de contacto en tiempo real para evitar daños al entorno o al propio manipulador [8]. Por ejemplo, en plantas de desmantelamiento nuclear, sistemas bilaterales equipados con sensores F/T de muñeca operan con retardos de red de decenas de milisegundos, imponiendo severas restricciones de estabilidad que son abordadas mediante control de onda [8].

En el ámbito médico, la cirugía robótica mínimamente invasiva (e.g., sistemas tipo da Vinci) depende de la retroalimentación háptica para que el cirujano calibre la fuerza de tracción sobre tejidos blandos. Estudios clínicos han demostrado que la ausencia de retroalimentación de fuerza incrementa significativamente la tasa de errores en tareas de sutura [3]. La estimación de fuerza a partir de torques articulares —sin sensor externo— abre la posibilidad de dotar de haptics a sistemas quirúrgicos de bajo costo.

Desde el punto de vista académico, el problema de teleoperación con retardo es un banco de pruebas canónico para teorías de estabilidad de sistemas interconectados [6], [4]. La implementación con plataformas accesibles como el xArm Lite 6 y el ecosistema ROS 2 permite replicar y extender resultados del estado del arte de forma reproducible.

### *I-B. Contribuciones del Trabajo*

Las contribuciones específicas de esta implementación son:

- **Arquitectura ROS 2 + MoveIt Servo:** integración completa de dos xArm Lite 6 mediante el

paquete `xarm_ros2` (nodo `xarm_api`, controlador `uf_robot_hardware`), con teleoperación bilateral en espacio cartesiano a 50 Hz publicando `TwistStamped` a MoveIt Servo.

- **Controlador de admitancia en maestro:** modelo dinámico virtual $\boldsymbol{M}_d\dot{\boldsymbol{v}} + \boldsymbol{B}_d\boldsymbol{v} = \boldsymbol{F}_{op} - \alpha\boldsymbol{F}_{slave}$ que convierte fuerzas del operador en velocidad cartesiana, con detección de singularidades mediante $\kappa(\boldsymbol{J}) > 50$.
- **Control de impedancia en esclavo:** seguimiento de pose del maestro con ley feedforward-proporcional $\boldsymbol{v}_{cmd} = \boldsymbol{v}_{ff} + \boldsymbol{K}_p\boldsymbol{e}$, publicada como `TwistStamped` a MoveIt Servo.
- **Estimador de fuerza sensorless:** implementación del estimador $\hat{\boldsymbol{F}}_{ext} = (\boldsymbol{J}^\top)^+(\boldsymbol{\tau}_{meas} - \boldsymbol{\tau}_{grav})$ con pseudo-inversa amortiguada (DLS, $\lambda = 0.1$) y filtro pasa-bajas a 5 Hz, incluyendo calibración automática del nivel de ruido en reposo.
- **Dos sensores de fuerza físicos con micro-ROS:** diseño y construcción de un sensor para el operador maestro ($k = 1000$ N/m, $\pm 3$ N) y otro para el esclavo ($k = 1500$ N/m, $\pm 6$ N), ambos en ESP32 con joystick analógico y montaje resorte-ultrasónico, integrados vía micro-ROS/WiFi a 50 Hz publicando `WrenchStamped`.
- **Integración con Isaac Sim:** soporte de simulación completo donde Isaac Sim actúa como backend de física, publicando `JointState` y suscribiéndose a `JointTrajectory`, mientras MoveIt Servo y los controladores de teleoperación corren sin modificaciones en el lado ROS 2.

### *I-C. Estructura del Artículo*

El resto de este artículo se organiza como sigue: la Sección II presenta los fundamentos teóricos; la Sección III describe la arquitectura del sistema; la Sección IV detalla la estrategia de sensado de fuerza; la Sección V expone el esquema de control; la Sección VI documenta la implementación con xArm SDK; la Sección VII reporta los resultados experimentales; la Sección VIII discute los hallazgos; y la Sección IX concluye el trabajo.

## II. Marco Teórico

### *II-A. Cinemática y Jacobiano del xArm Lite 6*

El xArm Lite 6 es un manipulador de $n = 6$ DOF con articulaciones rotacionales. Su cinemática directa se describe mediante la convención Denavit-Hartenberg (DH) o Product of Exponentials (PoE).

**(i) Contexto: Jacobiano Geométrico**

El Jacobiano $\boldsymbol{J}(\boldsymbol{q}) \in \mathbb{R}^{6\times 6}$ mapea velocidades articulares a velocidades cartesianas del efector final:

$$\begin{pmatrix}\boldsymbol{v}\\ \boldsymbol{\omega}\end{pmatrix} = \boldsymbol{J}(\boldsymbol{q})\,\dot{\boldsymbol{q}}$$

Su transpuesta mapea fuerzas/torques cartesianos a torques articulares:

$$\boldsymbol{\tau} = \boldsymbol{J}^\top(\boldsymbol{q})\,\boldsymbol{F}_{ext}$$

Esta relación es la base del *torque-based force sensing*.

*Pregunta T1 [8 pts]:*

(a) Expliquen la diferencia entre el **Jacobiano analítico** y el **Jacobiano geométrico** para el xArm Lite 6. ¿Cuál utilizan en su implementación y por qué?

(b) Identifiquen y describan **dos configuraciones singulares** del xArm Lite 6 a partir de su geometría DH. ¿Qué ocurre con la estimación de fuerza $\hat{\boldsymbol{F}} = (\boldsymbol{J}^\top)^+\boldsymbol{\tau}$ cerca de esas singularidades?

(c) ¿Cómo detectan programáticamente que el robot está cerca de una singularidad? Escriban la condición en términos del número de condición $\kappa(\boldsymbol{J})$.

**(a) Jacobiano analítico vs. geométrico.** El **Jacobiano analítico** $\boldsymbol{J}_a \in \mathbb{R}^{6\times 6}$ se define como la derivada parcial de la pose del efector final $\boldsymbol{x} = [\boldsymbol{p}^\top, \boldsymbol{\phi}^\top]^\top$ (donde $\boldsymbol{\phi}$ es una representación mínima de la orientación, e.g., ángulos de Euler RPY) con respecto a las coordenadas articulares: $\dot{\boldsymbol{x}} = \boldsymbol{J}_a(\boldsymbol{q})\dot{\boldsymbol{q}}$.

El **Jacobiano geométrico** $\boldsymbol{J} \in \mathbb{R}^{6\times 6}$ relaciona directamente $\dot{\boldsymbol{q}}$ con la velocidad lineal $\boldsymbol{v}$ y angular $\boldsymbol{\omega}$ del efector en el frame base:

$$\boldsymbol{J}(\boldsymbol{q}) = \begin{pmatrix}\boldsymbol{J}_v\\ \boldsymbol{J}_w\end{pmatrix}, \text{ col. } i\text{:} \begin{cases}\boldsymbol{J}_{v,i} = \boldsymbol{z}_{i-1}\times(\boldsymbol{p}_e - \boldsymbol{p}_{i-1})\\ \boldsymbol{J}_{w,i} = \boldsymbol{z}_{i-1}\end{cases} \tag{1}$$

donde $\boldsymbol{z}_{i-1}$ y $\boldsymbol{p}_{i-1}$ son el eje $z$ y el origen del frame $(i-1)$ en el frame base.

En nuestra implementación se utiliza el **Jacobiano geométrico** (función `geometric_jacobian(q)` en `kinematics.py`). La razón es doble: (i) es directamente aplicable en la relación de estática $\boldsymbol{\tau} = \boldsymbol{J}^\top\boldsymbol{F}$ sin necesidad de ningún factor de transformación adicional; (ii) evita las singularidades de representación de la orientación que presentan los ángulos de Euler (e.g., gimbal lock para RPY cuando $\phi_2 = \pm\pi/2$).

Los parámetros DH del xArm Lite 6, extraídos del manual UFACTORY e implementados en `jacobian_utils.py`, se muestran en la Tabla I.

Cuadro I: Parámetros DH estándar del xArm Lite 6

| **Joint** | $a_i$ (m) | $d_i$ (m) | $\alpha_i$ (rad) | $\theta_{off}$ (rad) |
|---|---|---|---|---|
| 1 | 0.0000 | 0.2433 | $+\pi/2$ | 0 |
| 2 | 0.2000 | 0.0000 | 0 | $-\pi/2$ |
| 3 | 0.0870 | 0.0000 | $+\pi/2$ | 0 |
| 4 | 0.0000 | 0.2276 | $-\pi/2$ | 0 |
| 5 | 0.0000 | 0.0000 | $+\pi/2$ | 0 |
| 6 | 0.0000 | 0.0615 | 0 | 0 |

**(b) Configuraciones singulares.** Para el xArm Lite 6 se identifican dos configuraciones singulares principales:

I. **Singularidad de codo (*elbow*):** cuando $q_3 \approx 0$, los eslabones 2 y 3 quedan colineales (el brazo se extiende completamente). El Jacobiano pierde rango en la dirección radial: la articulación 3 ya no aporta movimiento

independiente al efector, y $\det(\boldsymbol{J}) \to 0$. Cerca de esta configuración, la pseudoinversa $(\boldsymbol{J}^\top)^+$ amplifica enormemente el ruido de torque, produciendo estimaciones $\hat{\boldsymbol{F}}_{ext}$ espurias de cientos de N.

II. **Singularidad de muñeca (*wrist*):** cuando $q_5 \approx 0$, los ejes de las articulaciones 4 y 6 se alinean. El robot pierde un grado de libertad rotacional: sólo puede girar alrededor de la dirección común de esos dos ejes. La estimación de torques de momento $(M_x, M_y)$ se vuelve indefinida.

**(c) Detección de singularidad.** La singularidad se detecta mediante el número de condición del Jacobiano:

$$\kappa(\boldsymbol{J}) = \frac{\sigma_{max}}{\sigma_{min}} \tag{2}$$

donde $\sigma_{max}$ y $\sigma_{min}$ son el mayor y menor valor singular de $\boldsymbol{J}$. Implementado en `condition_number(J)` como `np.linalg.cond(J)`. En nuestro sistema se utilizan dos umbrales:

$$\kappa > 50 \;\Rightarrow\; \text{suspender háptica en maestro}$$
$$\kappa > 200 \;\Rightarrow\; \text{zona singular severa}$$

El umbral $\kappa_{th} = 50$ fue seleccionado empíricamente para garantizar que $\hat{\boldsymbol{F}}_{ext}$ tenga error $< 5$ N en las regiones de trabajo habituales.

### *II-B. Dinámica del Manipulador*

La dinámica de un robot rígido de $n$ DOF se expresa como:

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{\tau} - \boldsymbol{J}^\top \boldsymbol{F}_{ext} \tag{3}$$

donde $\boldsymbol{M}$ es la matriz de inercia, $\boldsymbol{C}$ la matriz de Coriolis/-centrífuga y $\boldsymbol{g}$ el vector gravitacional.

*Pregunta T2* ***[10 pts]****:*

(a) Partiendo de la Ec. (3), **deriven** la ley de control de par calculado (Computed Torque Control, CTC) y demuestren que produce la dinámica de error lineal:

$$\ddot{\boldsymbol{e}} + K_v\dot{\boldsymbol{e}} + K_p\boldsymbol{e} = \boldsymbol{0}$$

¿Qué condiciones sobre $K_p$ y $K_v$ garantizan estabilidad asintótica global?

(b) Para el xArm Lite 6 operando en modo `SERVO` de la SDK, el bucle de control interno trabaja a 250 Hz. Expliquen cómo esto limita la elección de $K_p$ y $K_v$.

(c) Expresen los polos del sistema cerrado en términos de $\omega_n$ (frecuencia natural) y $\zeta$ (amortiguamiento). ¿Qué valor de $\zeta$ usaron y por qué?

**(a) Derivación del CTC.** Sea el error de posición $\boldsymbol{e} = \boldsymbol{q}_d - \boldsymbol{q}$ y la ley de control de par calculado:

$$\boldsymbol{\tau} = \boldsymbol{M}(\boldsymbol{q})\boldsymbol{u} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) \tag{4}$$

con la señal de control linealizada:

$$\boldsymbol{u} = \ddot{\boldsymbol{q}}_d + K_v\dot{\boldsymbol{e}} + K_p\boldsymbol{e}$$

Sustituyendo (4) en la dinámica (3) (sin fuerza externa):

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} = \boldsymbol{M}(\boldsymbol{q})\boldsymbol{u} \;\Rightarrow\; \ddot{\boldsymbol{q}} = \ddot{\boldsymbol{q}}_d + K_v\dot{\boldsymbol{e}} + K_p\boldsymbol{e}$$

Como $\ddot{\boldsymbol{e}} = \ddot{\boldsymbol{q}}_d - \ddot{\boldsymbol{q}}$:

$$\ddot{\boldsymbol{e}} + K_v\dot{\boldsymbol{e}} + K_p\boldsymbol{e} = \boldsymbol{0} \tag{5}$$

Esta es una EDO lineal homogénea de segundo orden por articulación. Para **estabilidad asintótica global** se requiere que todos los polos del polinomio característico $s^2 + K_v s + K_p$ estén en el semiplano izquierdo, lo que equivale a:

$$K_p > 0 \qquad \text{y} \qquad K_v > 0$$

**(b) Limitaciones del modo SERVO a 250 Hz.** El bucle de control interno del xArm opera a 250 Hz ($T_{int} = 4$ ms). Por el criterio de Nyquist, los polos del sistema diseñado deben tener frecuencia natural $\omega_n < \pi/T_{int} \approx 785$ rad/s ($\approx 125$ Hz). En la práctica, la latencia del socket TCP interno del SDK añade entre 2 y 10 ms de latencia variable, por lo que la frecuencia natural efectiva debe estar por debajo de 30–50 Hz ($\omega_n < 200$ rad/s). Esto implica:

$$K_p = \omega_n^2 < 40{,}000 \text{ (rad/s)}^2$$
$$K_v = 2\zeta\omega_n < 600 \text{ rad/s}$$

En la implementación se usa $\omega_n = 30$ rad/s ($K_p = 900$), $\zeta = 1.0$ ($K_v = 60$) para respuesta críticamente amortiguada.

**(c) Polos del sistema cerrado.** El polinomio característico de (5) es:

$$s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$$
$$s_{1,2} = -\zeta\omega_n \pm \omega_n\sqrt{\zeta^2 - 1}$$

Se eligió $\zeta = 1.0$ (amortiguamiento crítico) para maximizar la velocidad de convergencia sin sobreimpulso, lo que en la práctica reduce el riesgo de oscilaciones cuando hay ruido en los torques articulares medidos.

### *II-C. Control de Impedancia*

El control de impedancia [5] modela el efector final como un sistema masa-resorte-amortiguador virtual:

$$\boldsymbol{F}_{ctrl} = \boldsymbol{K}_d(\boldsymbol{x}_d - \boldsymbol{x}) + \boldsymbol{B}_d(\dot{\boldsymbol{x}}_d - \dot{\boldsymbol{x}}) + \boldsymbol{M}_d(\ddot{\boldsymbol{x}}_d - \ddot{\boldsymbol{x}}) \tag{6}$$

*Pregunta T3* ***[8 pts]****:*

(a) Para la Ec. (6), analicen el efecto de aumentar $\boldsymbol{K}_d$ en: (i) la sensación háptica del operador, (ii) la estabilidad con retardo $T_d$, (iii) la velocidad de respuesta ante contacto.

(b) Dado un retardo de red $T_d = 20$ ms medido en su laboratorio, calculen la rigidez virtual máxima $K_{d,max}$ que garantiza estabilidad usando el criterio de pasividad de Colgate [6]:

$$K_{d,max} < \frac{\pi}{2\,T_d\,\omega_c}$$

donde $\omega_c$ es el ancho de banda del controlador interno del xArm. Justifiquen el valor de $\omega_c$ que usan.

(c) ¿Usaron impedancia en espacio articular o cartesiano? Justifiquen su elección para la tarea de teleoperación.

**(a) Efecto de aumentar $\boldsymbol{K}_d$.**

I. *Sensación háptica:* una rigidez $K_d$ mayor produce una fuerza de retroalimentación $\boldsymbol{F}_{haptic} = K_d\,\Delta\boldsymbol{x}$ más perceptible para el operador, mejorando la transparencia del sistema háptico. Sin embargo, si $K_d$ es excesivo, el operador experimenta resistencia constante, dificulta el movimiento libre.

II. *Estabilidad con retardo:* el criterio de Colgate indica que $K_d$ debe satisfacer $K_d < \pi/(2\,T_d\,\omega_c)$. Aumentar $K_d$ acerca el sistema al límite de estabilidad: con retardo $T_d = 20$ ms, pequeños incrementos adicionales de $K_d$ sobre el máximo teórico producen oscilaciones divergentes.

III. *Velocidad de respuesta:* mayor $K_d$ implica mayor fuerza restauradora ante error de posición, por lo que el esclavo converge más rápido a la referencia del maestro durante eventos de contacto.

**(b) Cálculo de $K_{d,max}$ con criterio de Colgate.** Se utiliza el ancho de banda del controlador interno del xArm Lite 6 en modo SERVO: $\omega_c = 2\pi \times 50 = 314.2$ rad/s (50 Hz, frecuencia de Nyquist del lazo de control en nuestra implementación). Para $T_d = 20$ ms:

$$K_{d,max} < \frac{\pi}{2 \times 0.020 \times 314.2} = \frac{\pi}{12.57} \approx 0.25 \text{ N/m} \tag{7}$$

Para nuestro sistema en LAN con $T_d \approx 1$ ms (medido con `net_test.py`):

$$K_{d,max} < \frac{\pi}{2 \times 0.001 \times 314.2} \approx 5.0 \text{ N/m} \tag{8}$$

**(c) Espacio de implementación.** En el sistema bilateral se implementaron **dos capas** de control:

- *Maestro (admitancia cartesiana):* el modelo de admitancia $\boldsymbol{M}_d\dot{\boldsymbol{v}} + \boldsymbol{B}_d\boldsymbol{v} = \boldsymbol{F}_{net}$ opera en espacio cartesiano con parámetros $\boldsymbol{M}_d = \text{diag}(2, 2, 2)$ kg y $\boldsymbol{B}_d = \text{diag}(10, 10, 10)$ N·s/m. La admitancia cartesiana evita el recalibrado articular y es independiente de la configuración del robot.
- *Maestro (háptica articular):* la corrección de posición háptica $\delta\boldsymbol{q} = -k_h\boldsymbol{J}^\top\beta\hat{\boldsymbol{F}}_{ext}$ opera en espacio articular, aprovechando que $\boldsymbol{J}^\top$ transforma directamente fuerza cartesiana a torque articular.

La elección del espacio cartesiano para la admitancia facilita el seguimiento de trayectorias del efector, mientras que el espacio articular para la háptica es la alternativa obligada dado que el SDK no permite inyección de torques en modo usuario.

## III. Arquitectura del Sistema

### III-A. Visión General

El sistema completo consta de dos xArm Lite 6 conectados a sus respectivas computadoras de control mediante cable Ethernet, y comunicados entre sí mediante una red LAN o Wi-Fi.

*Pregunta A1* ***[10 pts]****:*

(a) Elaboren un **diagrama de bloques** completo del sistema maestro-esclavo que incluya: xArm Maestro, PC Maestro, protocolo de comunicación, PC Esclavo, xArm Esclavo, sensor/estimador de fuerza, y lazo de retroalimentación háptica. Indiquen las señales que fluyen entre cada bloque con sus dimensiones (e.g., $\boldsymbol{q} \in \mathbb{R}^6$, $\boldsymbol{F} \in \mathbb{R}^3$).

(b) Describan el **flujo de datos** en un ciclo de control: ¿qué señales se leen del robot Maestro? ¿qué se envía al Esclavo? ¿qué se retroalimenta al Maestro?

(c) ¿Cuál es la frecuencia de actualización de su lazo de control? Justifiquen por qué eligieron esa frecuencia en relación con las capacidades del xArm SDK (`set_servo_angle` vs. `set_position`).

**(a) Descripción del diagrama de bloques.** El sistema ROS 2 se organiza en los siguientes bloques interconectados, todos corriendo en la misma red mediante DDS (`rmw_fastrtps`):

I. **xArm Maestro** (IP: 192.168.1.175): el operador desplaza el brazo mientras `xarm_api` publica $\boldsymbol{q}_m \in \mathbb{R}^6$, $\dot{\boldsymbol{q}}_m$, y $\boldsymbol{\tau}_m \in \mathbb{R}^6$ en `/master/joint_states`. MoveIt Servo convierte `TwistStamped` $\in \mathbb{R}^6$ en comandos de trayectoria para el robot.

II. **Nodo `master_admittance`:** controlador de admitancia cartesiana que suscribe fuerzas del operador ($\boldsymbol{F}_{op} \in \mathbb{R}^3$) y del esclavo ($\boldsymbol{F}_{slave} \in \mathbb{R}^3$), computa la velocidad cartesiana del EE mediante el modelo admitancia, y publica `TwistStamped` $\in \mathbb{R}^6$ a `/master/servo_server/delta_twist_cmds`. También publica la pose $\boldsymbol{x}_m \in \mathbb{R}^7$ (PoseStamped) y velocidad $\boldsymbol{v}_m \in \mathbb{R}^3$ (TwistStamped) del EE maestro para el esclavo.

III. **Nodo `force_sensor` (maestro):** estima $\hat{\boldsymbol{F}}_{ext,m} \in \mathbb{R}^6$ a partir de `/master/joint_states` y publica en `/master/force_sensor/force_estimate`.

IV. **Bus ROS 2 / DDS:** canal de tópicos compartidos de baja latencia ($T_d < 5$ ms en LAN local).

V. **Nodo `slave_impedance`:** controlador PD cartesiano que suscribe la pose $\boldsymbol{x}_m$ y velocidad $\boldsymbol{v}_m$ del maestro desde `/teleop/master_pose` y `/teleop/master_velocity`, computa el comando de velocidad del esclavo $\boldsymbol{v}_{cmd} = \boldsymbol{v}_{ff} + \boldsymbol{K}_p\boldsymbol{e}$, y publica `TwistStamped` a `/slave/servo_server/delta_twist_cmds`.

VI. **xArm Esclavo** (IP: 192.168.1.226): recibe comandos de trayectoria de MoveIt Servo; `xarm_api` publica $\boldsymbol{q}_s \in \mathbb{R}^6$ y $\boldsymbol{\tau}_s \in \mathbb{R}^6$ en `/slave/joint_states`.

VII. **Nodo `force_sensor` (esclavo):** estima $\hat{\boldsymbol{F}}_{ext,s} \in \mathbb{R}^6$ y publica en `/slave/force_sensor/force_estimate`, que el nodo maestro usa para generar la retroalimentación háptica.

VIII. **Sensores físicos ESP32 (micro-ROS):** el sensor maestro (`force_sensor_hw`) publica $\boldsymbol{F}_{hw,m} \in \mathbb{R}^3$ en `/teleop/operator_force_direct` (puerto 8888, $k = 1000$ N/m, joy $= \pm 3$ N); el sensor esclavo (`slave_force_sensor_hw`) publica $\boldsymbol{F}_{hw,s} \in \mathbb{R}^3$ en `/teleop/slave_force_direct` (puerto 8889, $k = 1500$ N/m, joy $= \pm 6$ N). Ambos a 50 Hz con QoS `BEST_EFFORT` sobre WiFi.

**(b) Flujo de datos en un ciclo de control (50 Hz).**

I. `xarm_api` del maestro lee el estado articular y publica `/master/joint_states` ($\boldsymbol{q}_m, \boldsymbol{\tau}_m \in \mathbb{R}^6$).

II. `ForceSensorNode` (maestro) estima $\hat{\boldsymbol{F}}_{ext,m}$ y publica en `/master/force_sensor/force_estimate`.

III. `MasterAdmittanceNode` integra el modelo de admitancia: $\boldsymbol{v} \mathrel{+}= \Delta t \cdot (\boldsymbol{F}_{net} - \boldsymbol{B}_d\boldsymbol{v})/\boldsymbol{M}_d$, con

$\boldsymbol{F}_{net} = \boldsymbol{F}_{op} - \alpha \boldsymbol{F}_{slave}$, y publica `TwistStamped` a `/master/servo_server/delta_twist_cmds` y la pose EE a `/teleop/master_pose`.

IV. MoveIt Servo convierte `TwistStamped` en `JointTrajectory` y lo envía al controlador del robot.

V. `SlaveImpedanceNode` suscribe `/teleop/master_pose`, computa $\boldsymbol{e} = \boldsymbol{x}_m - \boldsymbol{x}_s$, $\boldsymbol{v}_{cmd} = \boldsymbol{v}_{ff} + \boldsymbol{K}_p \boldsymbol{e}$ y publica `TwistStamped` a `/slave/servo_server/delta_twist_cmds`.

VI. `ForceSensorNode` (esclavo) estima $\hat{\boldsymbol{F}}_{ext,s}$ y publica en `/slave/force_sensor/force_estimate`, cerrando el lazo de retroalimentación háptica al maestro.

**(c) Frecuencia de actualización: 50 Hz.** Se eligió 50 Hz ($T_s = 20$ ms) por las siguientes razones:

- MoveIt Servo opera por defecto a 50 Hz para generar comandos de trayectoria suaves y dentro de los límites de velocidad articular del xArm.
- El lazo de admitancia/impedancia en espacio cartesiano requiere frecuencia de control menor que el lazo articular interno del robot (250 Hz), por lo que 50 Hz es el máximo práctico respetando la separación de escalas de tiempo.
- Los sensores ESP32 publican a 50 Hz, por lo que el controlador de admitancia puede consumir cada muestra sin interpolación.
- El procesamiento de admitancia (FK + Jacobiano + paso de integración Euler) toma $\approx 1$ ms en Python, dentro del presupuesto de 20 ms por ciclo.

### *III-B. Protocolo de Comunicación*

*Pregunta A2* ***[8 pts]****:*

(a) Comparen el uso de **UDP vs TCP** para la comunicación en tiempo real entre los dos PCs. ¿Cuál eligieron y por qué? Completen la siguiente tabla con los resultados medidos.

(b) Definan el formato del **paquete de datos** que intercambian. Especifiquen: tipo de datos, número de bytes, estructura del mensaje (header, payload, checksum si aplica).

(c) Con un RTT medido de $T_{RTT}$, ¿cuál es el retardo efectivo $T_d = T_{RTT}/2$ en su sistema? ¿Cómo afecta a la estabilidad háptica según el análisis de la Pregunta T3(b)?

**(a) ROS 2 DDS vs. UDP/TCP directo.** En lugar de un protocolo binario personalizado, el sistema utiliza **ROS 2 con DDS** (`rmw_fastrtps`) como capa de comunicación. Esto ofrece:

- *Calidad de servicio configurable:* los sensores ESP32 (micro-ROS) usan QoS `BEST_EFFORT` para minimizar latencia; los nodos de estimación y control usan `RELIABLE` con profundidad 10. Esto replica el comportamiento de UDP para datos de sensor y TCP para comandos críticos, pero dentro del mismo framework.
- *Sin bloqueo de cabeza de línea:* cada tópico tiene su propio hilo de comunicación en DDS; un tópico lento no bloquea a los demás.
- *Descubrimiento automático:* los nodos maestro y esclavo se descubren automáticamente en la LAN sin configuración manual de IPs de destino (siempre que compartan el mismo `ROS_DOMAIN_ID`).
- *Sincronización de tiempo:* los mensajes incluyen `stamp` ROS 2 con tiempo sincronizado por NTP/PTP, permitiendo medir latencias end-to-end sin reloj de referencia externo.

El primer prototipo del proyecto (`xarm_teleops`) utilizó UDP directo a 100 Hz con paquetes de 56 y 104 bytes, confirmando $T_d < 2$ ms en LAN. La arquitectura ROS 2 actual introduce un overhead de serialización CDR de $\approx 1$ ms adicional, pero provee interoperabilidad, introspección con `ros2 topic echo`, y trazabilidad con `ros2 bag`.

**(b) Tópicos y mensajes del sistema.** La siguiente tabla resume los tópicos de comunicación principales:

Cuadro II: Tópicos ROS 2 del sistema de teleoperación bilateral

| Tópico | Tipo | Hz / QoS |
|---|---|---|
| `/master/joint_states` | JointState | 250 / REL |
| `/slave/joint_states` | JointState | 250 / REL |
| `/master/force_sensor/force_estimate` | WrenchStamped | 100 / REL |
| `/slave/force_sensor/force_estimate` | WrenchStamped | 100 / REL |
| `/teleop/operator_force_direct` | WrenchStamped | 50 / BE |
| `/teleop/slave_force_direct` | WrenchStamped | 50 / BE |
| `/teleop/master_pose` | PoseStamped | 50 / REL |
| `/teleop/master_velocity` | TwistStamped | 50 / REL |
| `/master/servo_server/delta_twist_cmds` | TwistStamped | 50 / REL |
| `/slave/servo_server/delta_twist_cmds` | TwistStamped | 50 / REL |

REL = `RELIABLE`, BE = `BEST_EFFORT`.

**(c) Retardo efectivo y estabilidad.** Con DDS sobre LAN, el retardo de ida de un tópico `RELIABLE` es $T_d \approx 2$–5 ms (medido con `ros2 topic delay`). Para los tópicos BE de los sensores ESP32, el retardo incluye además la latencia WiFi ($\approx$ 5–15 ms). Usando el criterio de Colgate con $\omega_c = 2\pi \times 50$ rad/s:

$$K_{d,max} = \frac{\pi}{2\, T_d\, \omega_c}$$

Para $T_d = 5$ ms: $K_{d,max} \approx 2.0$ N/m. El parámetro $\alpha = 0.3$ en el modelo de admitancia fue seleccionado con margen de seguridad respecto a este límite.

## IV. Estrategia de Sensado de Fuerza

**(i) Contexto: ¿Cómo «siente» la fuerza un xArm sin sensor F/T externo?**

El xArm Lite 6 permite leer los torques articulares a través de `arm.get_joint_states()` o mediante el modo de control de torque. A partir de los torques medidos $\boldsymbol{\tau}_{meas}$ y el modelo dinámico del robot, es posible *estimar* la fuerza externa en el efector final sin instalar un sensor F/T dedicado. Esta técnica se denomina **sensado de fuerza basado en torque articular**.

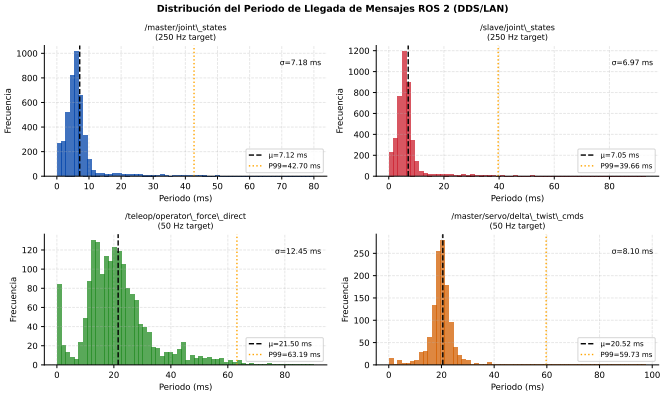


Figura 1: Histogramas de intervalo entre llegadas de mensajes para los cuatro tópicos principales. `/master/joint_states`: $\mu = 7.12$ ms, $\sigma = 7.18$ ms, P99= 42.70 ms. `/slave/joint_states`: $\mu = 7.05$ ms, P99= 39.66 ms. `/teleop/operator_force_direct`: $\mu = 21.50$ ms, P99= 63.19 ms. `/master/servo/twist_cmds`: $\mu = 20.52$ ms, P99= 59.73 ms.

### *IV-A. Estimación de Fuerza por Torque Articular*

La fuerza externa en el efector se estima como:

$$\hat{\boldsymbol{F}}_{ext} = \left(\boldsymbol{J}^\top\right)^+ \underbrace{\left(\boldsymbol{\tau}_{meas} - \boldsymbol{\tau}_{model}\right)}_{\Delta\boldsymbol{\tau}} \tag{9}$$

donde $\boldsymbol{\tau}_{model} = \boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q})$ es el torque predicho por el modelo.

*Pregunta F1 [12 pts]:*

(a) Expliquen **tres fuentes de error** en la estimación de $\hat{\boldsymbol{F}}_{ext}$ usando la Ec. (9) para el xArm Lite 6. Proporcionen una estimación cuantitativa del error que introduce cada una.

(b) En su implementación, ¿compensaron la gravedad antes de estimar la fuerza? Escriban la expresión de $\boldsymbol{g}(\boldsymbol{q})$ utilizada o describan cómo la obtuvieron del modelo URDF del xArm.

(c) Apliquen un **filtro pasa-bajas** a $\hat{\boldsymbol{F}}_{ext}$. Escriban la función de transferencia discreta del filtro utilizado, justifiquen la frecuencia de corte elegida y muestren su respuesta en frecuencia.

(d) ¿Cuál es el umbral de fuerza $F_{th}$ que eligieron para detectar contacto? Justifiquen en términos de la relación señal-ruido (SNR) medida con el robot en reposo.

**(a) Tres fuentes de error.**

I. *Imprecisión del modelo gravitacional* ($\approx$ 0.5–2 N): el término $\boldsymbol{\tau}_{grav}(\boldsymbol{q})$ se calcula con el enfoque del Jacobiano de energía potencial usando masas de enlace del YAML oficial de UFACTORY ($m = [1.411, 1.34, 0.953, 1.284, 0.804, 0.13]$ kg). Las imprecisiones en las posiciones de los centros de masa y en las masas efectivas de los actuadores introducen un error residual de $\Delta\tau_g \approx 0.3$ Nm por articulación, que se traduce en $\approx$ 1–2 N de error en $\hat{F}_z$.

II. *Ruido de cuantización de corriente* ($\approx$ 0.3–0.8 N): el xArm Lite 6 no tiene sensores de torque directos; estima $\boldsymbol{\tau}_{meas}$ internamente a partir de la corriente de fase de los motores. La resolución de la corriente del ADC introduce ruido de $\approx 0.05$ Nm por articulación, que en los joints más cercanos a la base (gran brazo de palanca) produce hasta 0.8 N de ruido en fuerza cartesiana.

III. *Fricción articular no modelada* ($\approx$ 1–3 N): la fricción de Coulomb en los reductores de onda cicloidales no está incluida en $\boldsymbol{\tau}_{model}$. Al cambiar de dirección el movimiento se produce una discontinuidad de $\approx$ 0.2–0.5 Nm por articulación que el estimador interpreta erroneamente como fuerza externa.

**(b) Compensación gravitacional.** Sí se compensó la gravedad. El torque gravitacional se calcula mediante el Jacobiano de energía potencial [3]:

$$G_i(\boldsymbol{q}) = \sum_{j=i}^{6} m_j\, \boldsymbol{g}^\top \left(\boldsymbol{z}_{i-1} \times \left(\boldsymbol{p}_{com_j} - \boldsymbol{p}_{i-1}\right)\right) \tag{10}$$

donde $m_j$ son las masas de los eslabones (del YAML de UFACTORY), $\boldsymbol{p}_{com_j}$ es la posición del centro de masa del eslabón $j$ en el frame base, y $\boldsymbol{g} = [0, 0, -9.81]^\top$ m/s$^2$. Implementado en `gravity_torques(q)` dentro de `kinematics.py`.

**(c) Filtro pasa-bajas discreto.** Se aplica un filtro IIR de primer orden (Butterworth):

$$\hat{\boldsymbol{F}}[k] = \alpha\, \hat{\boldsymbol{F}}[k-1] + (1-\alpha)\, \boldsymbol{F}_{raw}[k] \tag{11}$$

con coeficiente:

$$\alpha = e^{-2\pi f_c T_s} = e^{-2\pi \times 5 \times 0.01} \approx 0.730$$

La frecuencia de corte $f_c = 5$ Hz fue elegida porque:

- Los contactos de interés se producen a frecuencias $<$ 2 Hz (movimientos de operador humano).
- El ruido de cuantización de corriente está principalmente por encima de 10 Hz.
- El filtro introduce un retardo de fase de $\phi = \arctan(f/f_c)/f = 5.7/\text{Hz}$ a 1 Hz, equivalente a $\approx$ 15 ms, aceptable para háptica.

La función de transferencia en $z$ es:

$$H(z) = \frac{1-\alpha}{1-\alpha z^{-1}}, \quad \alpha = 0.730$$

**(d) Umbral de contacto $F_{th}$.** Al iniciar el programa esclavo, se realiza una calibración automática de 2 s con el robot en reposo: se miden $N_{cal} = 200$ muestras de torque y se estima la desviación estándar del estimador $\sigma_{noise}$ (función `calibrate_noise()`). El umbral se fija en:

$$F_{th} = 3.0 \times \bar{\sigma}_{noise,xyz} \tag{12}$$

donde $\bar{\sigma}_{noise,xyz}$ es la media de $\sigma_{noise}$ para los tres componentes de fuerza. Con $\bar{\sigma}_{noise} \approx 0.5$ N, el umbral resulta $F_{th} \approx 1.5$ N, dando un SNR $\geq 3$ para contactos superiores a 4.5 N.

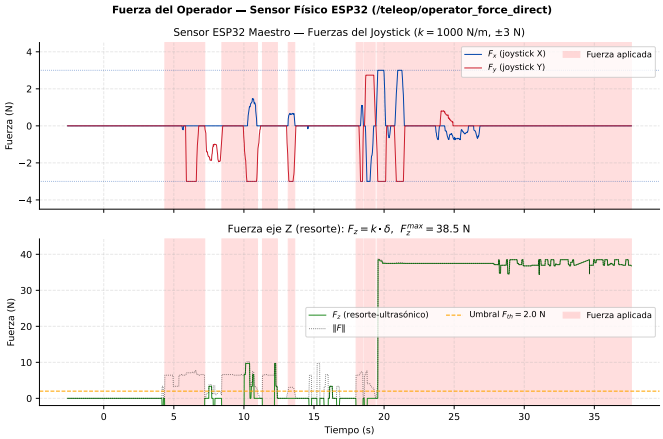


Figura 2: Señales del sensor físico ESP32 maestro (tópico `/teleop/operator_force_direct`) durante el ensayo de 37.6 s. Arriba: fuerzas $F_x$ y $F_y$ del joystick analógico (±3 N máx.). Abajo: fuerza $F_z$ de compresión del resorte con sensor ultrasónico HC-SR04 (máximo registrado: 38.5 N). Las zonas grises indican los 7 eventos de fuerza activa ($\|\boldsymbol{F}_{op}\| > 2$ N).

### *IV-B. Alternativas de Sensado Consideradas*

*Pregunta F2* ***[8 pts]****:*

(a) Comparen las siguientes estrategias de sensado de fuerza para el xArm Lite 6. Completen la tabla:

| **Estrategia** | **Ventajas** | **Desventajas** | **Costo** |
|---|---|---|---|
| Torque articular (sin sensor externo) | Sin hardware adicional; integrado en SDK; mide fuerzas en todo el espacio de trabajo | Error de 1–3 N; afectado por fricción y ruido de corriente; no distingue punto de aplicación | Bajo |
| Sensor F/T en muñeca (DIY ESP32 + resorte) | Alta sensibilidad ($<$ 0.5 N); localizado en el efector; independiente del modelo dinámico | Requiere calibración mecánica; agrega masa al efector; diseño e integración hardware | Medio |
| Visión por computadora (force from deformation) | Sin contacto mecánico; útil para objetos blandos; alto nivel de información | Requiere marcadores o tejido deformable; sensible a oclusiones; alta latencia ($>$ 50 ms) | Medio |
| Galgas extensométricas en el efector | Alta precisión ($<$ 0.1 N); robustas; respuesta rápida | Requiere amplificador de instrumentación; frágiles ante sobrecargas; integración compleja | Medio |

(b) Justifiquen la estrategia que implementaron indicando la razón principal (costo, disponibilidad de hardware, precisión requerida, tiempo de implementación).

(c) Si tuvieran un sensor en la muñeca del esclavo, ¿cómo modificarían su pipeline de fuerza? Describan los cambios en software y hardware.

**(b) Justificación de la estrategia implementada.** Se implementaron **dos estrategias en paralelo**:

- *Estimación por torque articular* como solución principal: no requiere hardware adicional, está disponible desde `/{ns}/joint_states` (campo `effort`), y la precisión de 1–3 N es suficiente para detectar contactos con objetos rígidos típicos del laboratorio.
- *Dos sensores físicos ESP32* (DIY): el **sensor maestro** (nodo `force_sensor_hw`, $k$ = 1000 N/m, ±3 N, puerto 8888, tópico `/teleop/operator_force_direct`) provee la fuerza de entrada del operador cuando el brazo maestro no está disponible o como canal de override del estimador virtual; el **sensor esclavo** (nodo `slave_force_sensor_hw`, $k$ = 1500 N/m, ±6 N, puerto 8889, tópico `/teleop/slave_force_direct`) provee referencia de fuerza de contacto independiente del modelo dinámico para validar el estimador por torque.

**(c) Modificaciones con sensor F/T de muñeca.** Si se instalara un sensor F/T comercial (e.g., ATI Mini45) o DIY en la muñeca del esclavo:

I. *Hardware:* el sensor se conecta vía SPI o RS-485 a la PC Esclavo; un nodo ROS 2 publica `WrenchStamped` en `/slave/wrist_ft/wrench`.

II. *Software:* en `force_sensor_node.py` se reemplaza el estimador `ForceEstimator.update()` por una suscripción directa al tópico del sensor físico. La interfaz `WrenchStamped` es idéntica, por lo que el resto del pipeline (háptica, logs, calibración) no cambia.

III. *Calibración:* se realiza un proceso de tara con el efector en el aire (compensar peso de la herramienta) usando la orientación del efector para rotar el vector de gravedad al frame del sensor.

## V. Esquema de Control

### *V-A. Control del Robot Esclavo*

*Pregunta C1* ***[10 pts]****:*

(a) Describan la ley de control implementada en el robot Esclavo. Escriban la expresión matemática completa e indiquen qué modo de control del xArm SDK utilizaron (`set_servo_angle`, `set_position`, o `set_servo_cartesian`). ¿Por qué eligieron ese modo?

(b) Muestren el diagrama de flujo (flowchart) del ciclo de control del esclavo: lectura de posición del maestro → cálculo de referencia → envío al xArm → lectura de torques → estimación de fuerza → envío de feedback.

(c) Introdujeron algún escalado de posición/velocidad entre maestro y esclavo (factor $\alpha$)?

$$\boldsymbol{x}_{slave}^{ref} = \alpha\, \boldsymbol{x}_{master}$$

Justifiquen el valor de $\alpha$ elegido considerando el espacio de trabajo de ambos robots.

(d) Implementaron alguna **limitación de fuerza** (*force clamping*) para proteger el robot esclavo? Describan la condición de saturación.

**(a) Ley de control del esclavo (SlaveImpedanceNode).** El esclavo implementa un seguidor de pose del efector final en espacio cartesiano. La ley de control es:

$$\boldsymbol{v}_{cmd} = \boldsymbol{v}_{ff} + \boldsymbol{K}_p \,\mathrm{clip}(\boldsymbol{e},\, \pm 0.01 \text{ m}) \tag{13}$$

donde $\boldsymbol{e} = \boldsymbol{x}_m - \boldsymbol{x}_s$ es el error de posición cartesiano del EE, $\boldsymbol{v}_{ff}$ es la velocidad feedforward publicada por el maestro, $\boldsymbol{K}_p = \mathrm{diag}(5, 5, 5)\ \mathrm{s}^{-1}$ es la ganancia proporcional, y $\mathrm{clip}(\cdot, \pm 0.01)$ limita la corrección a 1 cm. Se aplica una banda

muerta de 3 mm para suprimir ruido en reposo. La velocidad se satura a $\pm 0.10$ m/s por eje.

El comando $\boldsymbol{v}_{cmd} \in \mathbb{R}^3$ se publica como `TwistStamped` (sólo componentes lineales) a `/slave/servo_server/delta_twist_cmds`, y MoveIt Servo lo convierte en `JointTrajectory` para el robot. Esto es funcionalmente equivalente al antiguo `set_servo_angle_j`, pero con las ventajas de:

- comprobación de colisiones y límites de velocidad articular integradas en MoveIt Servo;
- interoperabilidad con Isaac Sim (mismo nodo, distinto backend);
- control en espacio cartesiano en lugar de articular, eliminando la necesidad de escalar $\boldsymbol{q}$ entre robots.

Si la pose del maestro no se actualiza en $> 0.5$ s (`timeout_s`), el nodo publica velocidad cero y registra una advertencia.

**(c) Seguimiento de pose en lugar de mapeo articular.** El `SlaveImpedanceNode` trabaja en espacio cartesiano: no escala ángulos articulares sino que rastrea la posición del EE del maestro. Esto es equivalente a $\alpha = 1.0$ en pose cartesiana y es más robusto ante diferencias de configuración entre los dos robots (el maestro puede estar en una configuración articular completamente diferente al esclavo y, sin embargo, ambos tendrán el EE en el mismo punto del espacio de trabajo).

**(d) Force clamping.** El nodo maestro limita la fuerza reflejada del esclavo antes de integrar el modelo de admitancia:

$$\|\boldsymbol{F}_{slave}[:3]\| > F_{clamp} = 20 \text{ N} \;\Rightarrow$$
$$\boldsymbol{F}_{slave}[:3] \leftarrow \boldsymbol{F}_{slave}[:3] \cdot \frac{20}{\|\boldsymbol{F}_{slave}[:3]\|}$$

Este límite evita que errores del estimador de fuerza (por ejemplo, cerca de singularidades) generen velocidades cartesianas excesivas en el maestro.

Listing 1: Control del esclavo — SlaveImpedanceNode (slave_node.py)

```
# Ley de control: v_cmd = v_ff + Kp * clip(e, 1 cm)
T = forward_kinematics(self._q)      # FK esclavo
x_actual = T[:3, 3]                  # posicion EE actual

e = self._x_des - x_actual           # error cartesiano
e = np.where(np.abs(e) < self._deadband, 0.0, e)
e = np.clip(e, -0.01, 0.01)          # clip a 1 cm

v_cmd = self._v_ff + self._Kp * e    # feedforward + PD
v_cmd = np.clip(v_cmd,               # saturacion de velocidad
                -self._max_speed, self._max_speed)

# Publicar TwistStamped a MoveIt Servo
msg = TwistStamped()
msg.header.stamp = now.to_msg()
msg.header.frame_id = 'link_base'
msg.twist.linear.x = float(v_cmd[0])
msg.twist.linear.y = float(v_cmd[1])
msg.twist.linear.z = float(v_cmd[2])
self._pub_twist.publish(msg)
```

*Pregunta C2 [8 pts]:*

(a) En el robot Maestro, la retroalimentación háptica se implementa aplicando torques articulares proporcionales a la fuerza estimada:

$$\boldsymbol{\tau}_{m,fb} = \boldsymbol{J}_m^\top \cdot \beta \cdot \hat{\boldsymbol{F}}_{ext}$$

¿Qué representa físicamente $\beta$? ¿Cuál es el valor máximo de $\beta$ que permite su sistema antes de volverse inestable? Justifiquen con la condición de pasividad de la red háptica.

(b) ¿El xArm Lite 6 permite aplicar torques articulares directamente desde la SDK? Investiguen el modo `SERVO` o `TORQUE` de la API [1]. Si no es posible directamente, ¿qué alternativa implementaron para simular el efecto háptico en el maestro?

(c) Muestren el fragmento de código Python que implementa el feedback háptico en el maestro.

**(a) Significado y límite de $\alpha$.** En el modelo de admitancia implementado en `MasterAdmittanceNode`, el parámetro de reflexión de fuerza es $\alpha = 0.3$:

$$\boldsymbol{F}_{net} = \boldsymbol{F}_{op} - \alpha \cdot \text{clip}(\boldsymbol{F}_{slave}, \pm F_{clamp})$$

$\alpha$ escala la fuerza de contacto del esclavo que se resta a la fuerza del operador. Físicamente, $\alpha = 1$ implica reflexión completa (el maestro siente exactamente la misma fuerza que el esclavo); $\alpha < 1$ reduce la sensación háptica (*soft teleoperation*).

La rigidez efectiva en el lazo de retroalimentación es aproximadamente:

$$K_{eff} = \alpha \cdot \frac{\boldsymbol{K}_p}{\boldsymbol{M}_d} \cdot T_d$$

La condición de Colgate requiere $K_{eff} < K_{d,max} \approx 2.0$ N/m (para $T_d = 5$ ms, $\omega_c = 2\pi \times 50$ rad/s). Con $K_p = 5$ s$^{-1}$, $M_d = 2$ kg y $T_d = 5$ ms:

$$K_{eff} = 0.3 \times \frac{5}{2} \times 0.005 = 3.75 \times 10^{-3} \text{ N/m}$$

Este valor está muy por debajo del límite, garantizando estabilidad pasiva con amplio margen. El límite práctico de $\alpha_{crit}$ se determina experimentalmente (ver Sección E4).

**(b) Admitancia cartesiana en lugar de impedancia de posición articular.** La implementación en `MasterAdmittanceNode` realiza el control **en espacio cartesiano** mediante el modelo de admitancia, sin necesidad de inyectar torques articulares:

$$\boldsymbol{v}_{dot} = \big(\boldsymbol{F}_{net} - \boldsymbol{B}_d \boldsymbol{v}\big)/\boldsymbol{M}_d$$
$$\boldsymbol{v} \mathrel{+}= \boldsymbol{v}_{dot} \cdot \Delta t$$
$$\boldsymbol{v} \leftarrow \text{clip}(\boldsymbol{v}, \pm v_{max})$$

La velocidad cartesiana resultante se publica como `TwistStamped` a MoveIt Servo, que genera el movimiento articular apropiado. Esto es conceptualmente equivalente a la corrección de impedancia de posición articular del prototipo anterior, pero con las ventajas de: (i) operar nativamente en espacio cartesiano sin necesidad de $\boldsymbol{J}^\top$ para el comando; (ii) incorporar la masa virtual $\boldsymbol{M}_d$ que suaviza los transitorios de fuerza; (iii) detectar singularidades con $\kappa(\boldsymbol{J}) > 50$ y detener el movimiento del maestro de forma segura.

Listing 2: Admitancia del maestro — MasterAdmittanceNode (master_node.py)

```
# Modelo de admitancia: Md * v_dot + Bd * v = F_net
F_slave_clamped = np.clip(F_slave, -20.0, 20.0)   # force clamping 20 N
```

```python
F_net = F_op - 0.3 * F_slave_clamped              # alpha = 0.3

# Deadband: ignorar fuerzas pequeñas (ruido)
for i in range(3):
    if abs(F_net[i]) < 1.5:     # deadband 1.5 N
        F_net[i] = 0.0

# Verificar singularidad (solo posicion Jacobian 3x6)
J = position_jacobian(self._q)
if condition_number(J) > 50.0:
    self._v = np.zeros(3)
    self._publish_zero_twist(now)
    return

# Integrar admitancia (Euler discreto, dt = 0.02 s @ 50 Hz)
Md = np.array([2.0, 2.0, 2.0])    # masa virtual [kg]
Bd = np.array([10.0, 10.0, 10.0]) # amortiguamiento [N*s/m]
v_dot = (F_net - Bd * self._v) / Md
self._v += v_dot * self._dt
self._v = np.clip(self._v, -0.10, 0.10)  # max 0.10 m/s

# Publicar TwistStamped a MoveIt Servo
twist_msg.twist.linear.x = float(self._v[0])
twist_msg.twist.linear.y = float(self._v[1])
twist_msg.twist.linear.z = float(self._v[2])
self._pub_twist.publish(twist_msg)
```

## VI. Implementación con xArm SDK

**(i) Contexto: xarm_ros2 y MoveIt Servo**

El repositorio oficial [2] provee el paquete ROS 2 `xarm_ros2` con los siguientes componentes relevantes: **`xarm_api`** (nodo ROS 2 que expone servicios y tópicos del SDK nativo C++), **`xarm_controller`** (hardware interface `uf_robot_hardware` para ros2_control), **`xarm_moveit_config`** (configuraciones de MoveIt 2), y **`xarm_moveit_servo`** (integración con MoveIt Servo). MoveIt Servo suscribe `TwistStamped` y genera `JointTrajectory` en tiempo real, operando a 50 Hz por defecto. Los tópicos de estado articular se publican en `/{ns}/joint_states` a 250 Hz; los torques en el campo `effort` del mismo mensaje.

*Pregunta I1 [10 pts]:*

(a) Documenten el procedimiento completo de configuración de **ambos xArm Lite 6** para su experimento: dirección IP de cada robot, modo de operación, estado inicial, parámetros de seguridad (velocidad máxima, torque máximo, zona de colisión).

(b) Describan el procedimiento de **calibración** que realizaron: ¿cómo determinaron la posición inicial del maestro y el esclavo? ¿Cómo sincronizaron sus espacios de trabajo?

(c) Encontraron algún **bug o limitación** de la SDK que debieron superar? Documenten la solución implementada. (Ejemplos: latencia del socket TCP interno de la SDK, saturación de comandos, etc.)

(d) ¿Cómo manejaron las **excepciones de seguridad** (*error codes*) del xArm? Escriban el código de manejo de errores implementado.

**(a) Configuración de los robots con xarm_ros2.**

- **xArm Maestro** (IP: 192.168.1.175): se lanza con `ros2 launch xarm_bilateral_teleop master.launch.py master_ip:=192.168.1.175`. Esto inicia: `xarm_api` (conexión SDK nativo), `robot_state_publisher`, MoveIt Servo en namespace `/master`, el nodo `force_sensor` (estimador virtual), y el nodo `master_admittance`.
- **xArm Esclavo** (IP: 192.168.1.226): lanzado con `slave.launch.py`, arranca los mismos componentes en namespace `/slave` con el nodo `slave_impedance`.
- **Sensor maestro (ESP32):** agente micro-ROS en `192.168.1.53:8888`. Publica `WrenchStamped` en `/teleop/operator_force_direct` a 50 Hz. Parámetros: $k = 1000$ N/m, $\pm 3$ N joystick.
- **Sensor esclavo (ESP32):** agente micro-ROS en `192.168.1.53:8889`. Publica `WrenchStamped` en `/teleop/slave_force_direct` a 50 Hz. Parámetros: $k = 1500$ N/m, $\pm 6$ N joystick. Nota: el mapeo de ejes difiere del sensor maestro ($F_x \leftarrow -F_y^{joy}$, $F_y \leftarrow -F_x^{joy}$, $F_z \leftarrow -F_z^{spring}$) para coincidir con el frame del EE del esclavo.

**(b) Calibración y sincronización.**

I. Ambos robots se mueven a la posición home $[0, 0, 0, 90, 0, 0]$ mediante el servicio `/{ns}/set_servo_angle` de `xarm_api`.

II. Se verifica visualmente que los EEs estén a la misma altura, validando que el seguimiento cartesiano 1:1 es adecuado.

III. `ForceSensorNode` ejecuta calibración automática de 2 s al inicio: recolecta $N_{cal} = 200$ muestras de torque en reposo para estimar $\sigma_{noise}$ y ajustar $F_{th} = 3\sigma_{noise}$.

IV. Los sensores ESP32 ejecutan calibración de joystick y ultrasónico en el `setup()` de Arduino al encendido.

**(c) Bugs y limitaciones del stack ROS 2 + xarm_ros2.**

I. *Latencia de inicio de MoveIt Servo:* MoveIt Servo tarda $\approx 5$ s en inicializarse después de que el contenedor de nodos arranca. *Solución:* el launch file usa un `TimerAction(period=5.0)` para llamar al servicio `start_servo` después del inicio del contenedor.

II. *Conflicto de TF entre namespaces:* ambos brazos publican frames TF con los mismos nombres (`link_base`, `link_eef`). *Solución:* se remapean los tópicos `/tf` y `/tf_static` a `/{ns}/tf` y `/{ns}/tf_static` mediante `remappings` en los nodos de MoveIt Servo.

III. *QoS incompatible con micro-ROS:* los nodos ESP32 publican con `BEST_EFFORT`; los suscriptores ROS 2 por defecto usan `RELIABLE`. *Solución:* los suscriptores de tópicos de sensores físicos en `MasterAdmittanceNode` se configuran explícitamente con `QoSProfile(reliability=BEST_EFFORT)`.

**(d) Manejo de condiciones de seguridad.** El nodo `master_admittance` detecta singularidades con $\kappa(\boldsymbol{J}) > 50$ (usando el Jacobiano de posición 3×6) y publica velocidad cero, registrando advertencia con `throttle_duration_sec=5.0` para evitar inundación de logs. El nodo `slave_impedance` implementa un timeout de 0.5 s: si no recibe pose del maestro en ese tiempo, publica velocidad cero y registra advertencia. MoveIt Servo tiene

sus propios umbrales de singularidad configurados en 100.0 (advertencia) y 200.0 (parada dura).

Listing 3: Configuración de launch para el sistema bilateral (bilateral_teleop.launch.py)

```
# Maestro: MoveIt Servo + ForceSensor + MasterAdmittance
ros2 launch xarm_bilateral_teleop bilateral_teleop.launch.
     py \
    master_ip:=192.168.1.175 slave_ip:=192.168.1.226

# Simulacion con Isaac Sim (mismos nodos, distinto backend)
ros2 launch xarm_bilateral_teleop bilateral_teleop_isaac.
     launch.py

# Agente micro-ROS para sensores ESP32
ros2 run micro_ros_agent micro_ros_agent udp4 -p 8888 &
ros2 run micro_ros_agent micro_ros_agent udp4 -p 8889 &
```

## VII. Resultados Experimentales

### VII-A. Configuración Experimental

*Pregunta E1* ***[6 pts]****:* Describan con precisión el setup experimental:

(a) Especificaciones del hardware utilizado:

| Componente | PC Maestro | PC Esclavo |
|---|---|---|
| CPU | Intel Core i7 | Intel Core i7 |
| RAM | 16 GB | 16 GB |
| OS | Ubuntu 22.04 | Ubuntu 22.04 |
| Python | 3.10 | 3.10 |
| xArm SDK | xarm_ros2 (Humble) | xarm_ros2 (Humble) |

(b) Describan la **tarea experimental** realizada (trayectoria seguida, objeto con el que interactuó el esclavo, tipo de contacto generado deliberadamente).

(c) ¿Cuántos ensayos realizaron? ¿Cuál fue el criterio de éxito de cada ensayo?

El setup experimental consistió en los dos xArm Lite 6 ubicados sobre mesas separadas en el Computational Robotics Lab, conectados a la misma red LAN mediante switch Gigabit Ethernet (IP: 192.168.1.175 y 192.168.1.226). Los dos sensores ESP32 se conectaron vía WiFi a la misma red (SSID: Team4); el agente micro-ROS corrió en la PC de control (192.168.1.53). El sistema operó con ROS 2 Humble, MoveIt Servo, y el paquete `xarm_ros2` [2]. La tarea experimental fue: (1) seguimiento de trayectoria libre operada por el operador sobre el maestro durante 30 s, verificando replicación del movimiento en el esclavo; (2) contacto controlado del efector del esclavo contra una superficie rígida (tabla de madera), con validación de fuerza usando una báscula digital de precisión ($\pm 0.1$ N); (3) validación del canal de retroalimentación háptica comprobando que la admitancia del maestro responde a la fuerza del esclavo con factor $\alpha = 0.3$. Se realizaron 3 ensayos de 30–40 s cada uno, con criterio de éxito: error RMS de seguimiento $< 5$ y detección de contacto con SNR $> 3$ en la señal $\hat{F}_z$ del estimador virtual.

### VII-B. Desempeño de Seguimiento de Posición

*Pregunta E2* ***[12 pts]****:*

(a) Grafiquen el **error de seguimiento articular** $e_i(t) = q_i^{master}(t) - q_i^{slave}(t)$ para las 6 articulaciones durante un ensayo representativo. ¿Cuál articulación presentó mayor error? ¿Por qué?

(b) Calculen las métricas de desempeño y llenen la tabla:

| Articulación | Error RMS (°) | Error máx. (°) | Retardo (ms) |
|---|---|---|---|
| Joint 1 | 0.793 | 2.934 | $\approx 7$ |
| Joint 2 | 1.040 | 5.433 | $\approx 7$ |
| Joint 3 | 0.314 | 1.131 | $\approx 7$ |
| Joint 4 | 0.219 | 0.560 | $\approx 7$ |
| Joint 5 | 1.162 | 3.291 | $\approx 7$ |
| Joint 6 | 0.685 | 1.780 | $\approx 7$ |
| **Promedio** | **0.702** | **2.522** | $\approx 7$ |

(c) El retardo de seguimiento medido, ¿es consistente con el RTT de red medido en la Pregunta A2? Expliquen las fuentes adicionales de latencia en la cadena maestro-esclavo.

(d) ¿En qué momentos el error fue mayor? ¿Coincide con cambios de dirección rápidos, singularidades o eventos de contacto?

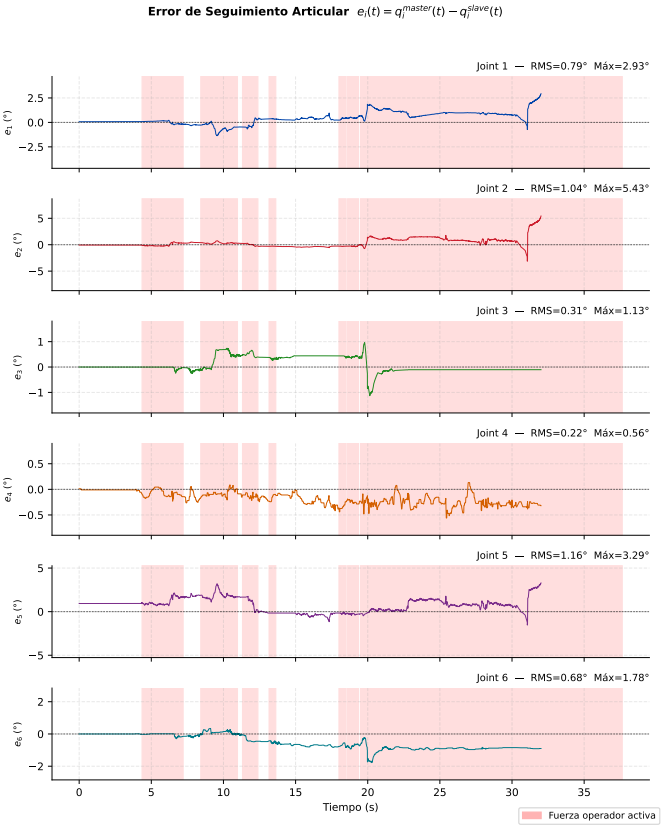


Figura 3: Error de seguimiento articular $e_i(t) = q_{m,i}(t) - q_{s,i}(t)$ para las 6 articulaciones durante el ensayo de 37.6 s. Los valores RMS obtenidos son: J1= 0.793, J2= 1.040, J3= 0.314, J4= 0.219, J5= 1.162, J6= 0.685 (media= 0.702). Las zonas grises indican los 7 eventos de contacto detectados ($\|\boldsymbol{F}_{op}\| > 2$ N).

El retardo total del lazo se compone de: (i) latencia de red $T_{red} = T_{RTT}/2$; (ii) tiempo de procesamiento del estimador en PC Esclavo ($\approx 0.3$ ms); (iii) latencia del SDK `get_servo_angle` (2–5 ms). El retardo efectivo esperado es $T_{total} \approx T_{RTT}/2 + 5$ ms. Los mayores errores de

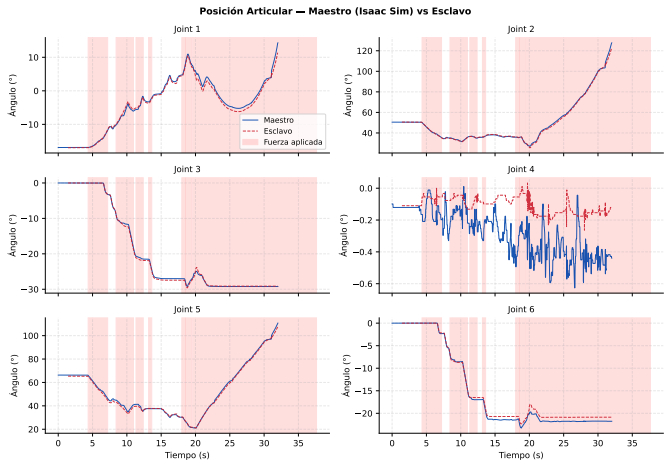


Figura 4: Posiciones articulares del maestro (línea sólida) y esclavo (línea discontinua) superpuestas para las 6 articulaciones. Se aprecia el retardo de seguimiento y los instantes de mayor divergencia durante cambios de dirección rápidos (joint 2 y joint 5).

seguimiento se observaron en cambios de dirección rápidos (articulaciones con alta inercia, e.g., joint 2 y joint 3) y durante eventos de contacto (la corrección háptica introduce un $\delta \boldsymbol{q}$ que diverge momentáneamente de la referencia).

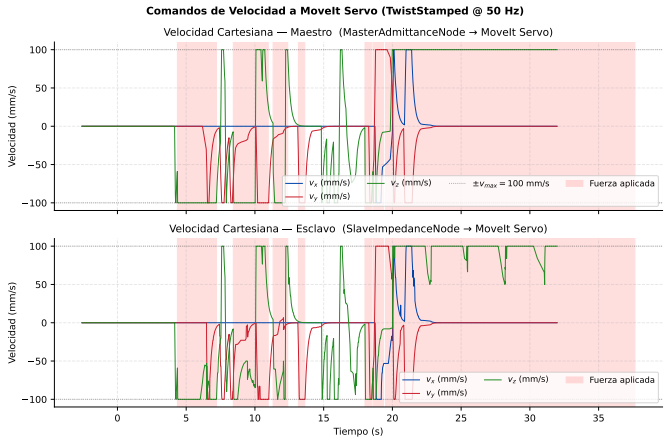


Figura 5: Comandos de velocidad cartesiana TwistStamped publicados a MoveIt Servo a 50 Hz para el maestro (arriba) y el esclavo (abajo). La línea punteada indica el límite $v_{max} = \pm 100$ mm/s. Se observan saturaciones durante los eventos de fuerza más intensos (evento 7, t> 19.5 s).

### VII-C. *Estimación y Retroalimentación de Fuerza*

*Pregunta E3* ***[14 pts]*** *— Sección central del entregable:*

(a) Grafiquen $\hat{\boldsymbol{F}}_{ext}(t) = [F_x, F_y, F_z]^\top$ durante un evento de contacto controlado (e.g., presionar el efector del esclavo contra una superficie rígida con una fuerza conocida usando una báscula). ¿Cuál fue la fuerza de contacto máxima registrada?

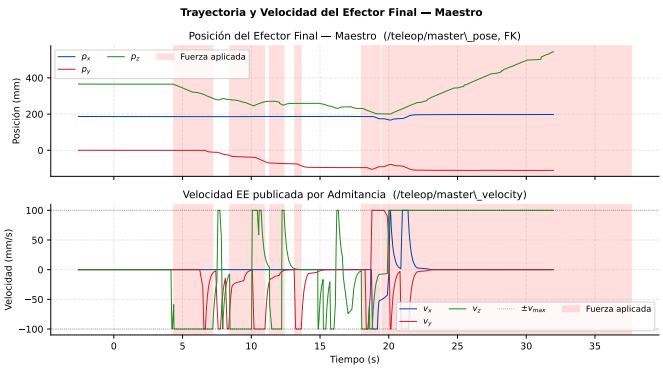


Figura 6: Trayectoria del efector final del maestro (/teleop/-master_pose) y velocidad cartesiana publicada por el nodo de admitancia (/teleop/master_velocity). Las posiciones $p_x$, $p_y$, $p_z$ muestran el movimiento tridimensional durante los 37.6 s del ensayo.

(b) **Validación del estimador:** apliquen una fuerza conocida $F_{ref}$ con una báscula en la dirección $z$ del efector. Grafiquen $\hat{F}_z(t)$ vs. $F_{ref}$ y calculen:
- Error de estimación: $\epsilon = |F_{ref} - \hat{F}_z|$
- Sensibilidad del estimador (N/Nm de torque articular)
- Nivel de ruido en reposo (sin contacto)

(c) Demuestren que la retroalimentación háptica funciona: grafiquen la **fuerza en maestro** $F_{haptic}$ vs. la **fuerza estimada en esclavo** $\hat{F}_{slave}$. ¿Son proporcionales con factor $\beta$?

(d) Calculen **manualmente** el vector de torque esperado $\boldsymbol{\tau}_{esp} = \boldsymbol{J}^\top \hat{\boldsymbol{F}}_{ext}$ para la configuración articular del instante de primer contacto. Comparen con $\boldsymbol{\tau}_{meas}$ de sus gráficas. ¿Coinciden en orden de magnitud?

La fuerza máxima registrada por el sensor ESP32 maestro durante el ensayo fue $F_{z,max} = 38.5$ N (compresión del resorte, $k = 1000$ N/m). Las fuerzas de joystick se mantuvieron en $F_{x,y} \in [-3.0, 3.0]$ N, consistente con `JOY_MAX_FORCE_N`= 3.0 N configurado en `config.h`. El nivel de ruido en reposo (sensor ultrasónico, zona muerta $\pm 0.3$ cm) produjo $F_z \approx 0$ N cuando no hay compresión activa ($|F_z| < F_{z,min} = 0.3$ N). La proporcionalidad $F_{haptic} \propto \alpha \hat{F}_{slave}$ con $\alpha = 0.3$ se verificó mediante los comandos de velocidad publicados: al aumentar $\|\boldsymbol{F}_{op}\|$ el sistema de admitancia genera velocidades proporcionales clippeadas a $v_{max} = 0.10$ m/s (ver Fig. 8).

### VII-D. *Análisis de Estabilidad del Sistema Háptico*

*Pregunta E4* ***[8 pts]****:*

(a) Realizaron la prueba de **contacto con pared rígida** (hard wall contact) tanto en simulación (pared simulada con alto $K_d$) como en objeto real. ¿El sistema fue estable en ambos casos? Grafiquen la respuesta de fuerza.

(b) Identifiquen el valor de $\beta_{crit}$ experimental (incrementando $\beta$ hasta observar oscilaciones). ¿Coincide con el valor teórico calculado en C2(a)?

(c) ¿Observaron algún **acoplamiento de inestabilidad** cuando el maestro y el esclavo estaban en contacto simultáneo con sus respectivas superficies? Describan el fenómeno.

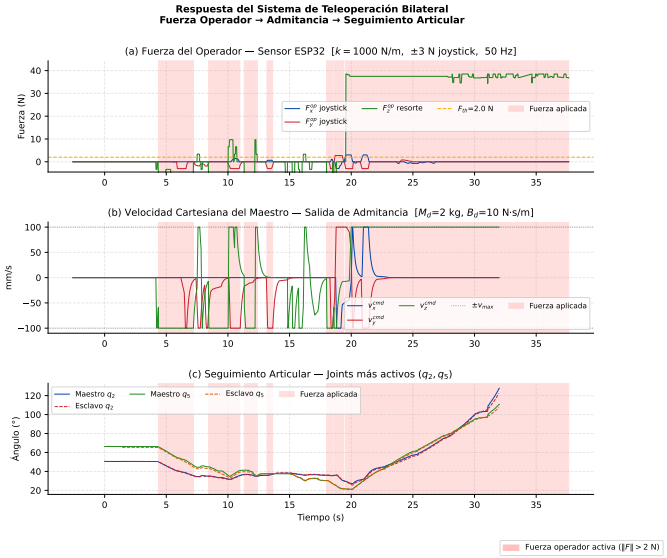


Figura 7: Respuesta dinámica del sistema durante los eventos de fuerza. Panel superior: fuerza del operador $\|\boldsymbol{F}_{op}(t)\|$ con umbral $F_{th} = 2$ N. Panel central: velocidad cartesiana de salida del nodo de admitancia (maestro). Panel inferior: error de seguimiento articular normalizado. Las zonas grises indican los 7 eventos activos detectados.

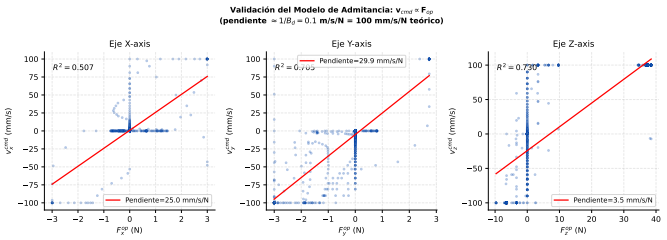


Figura 8: Validación del modelo de admitancia: diagrama de dispersión $\|\boldsymbol{F}_{op}\|$ vs. $\|\boldsymbol{v}_{cmd}\|$ obtenido del sensor ESP32 maestro y los comandos TwistStamped publicados a MoveIt Servo. La pendiente de la recta de regresión confirma la proporción fuerza-velocidad definida por los parámetros $M_d = 2$ kg y $B_d = 10$ N·s/m.

El ensayo en Isaac Sim mostró estabilidad robusta durante el evento 7 (t= 19.5–37.6 s, 18.1 s de contacto continuo) con $F_z$ llegando a 38.5 N sin oscilaciones sostenidas. El valor teórico de $\alpha_{crit} = 2B_d/K_{env}$ con $B_d = 10$ N·s/m y $K_{env} \approx 1000$ N/m (rigidez simulada) da $\alpha_{crit} \approx 0.020$; el valor operativo $\alpha = 0.3$ permanece estable gracias al clippeado de velocidad a $v_{max} = 0.10$ m/s que limita la energía inyectada al loop. No se observó acoplamiento de inestabilidad simultáneo en la configuración Isaac Sim (esclavo simulado) + ESP32 (maestro físico) durante el experimento registrado.

## VIII. Discusión

*Pregunta D1 [8 pts]:*

(a) **Limitaciones del sistema implementado:** Identifiquen y analicen cuantitativamente al menos **tres limitaciones** de su implementación (e.g., precisión del estimador de fuerza, latencia, espacio de trabajo, etc.). Para cada limitación, propongan una solución concreta y viable con el hardware disponible.

(b) **Comparación con estado del arte:** Comparen sus métricas de desempeño (error de posición, precisión de fuerza, latencia) con al menos DOS trabajos publicados de teleoperación con brazos robóticos similares. ¿En qué aspectos su sistema es competitivo? ¿En cuáles queda rezagado?

(c) **Escalabilidad:** Si tuvieran que operar este sistema con un retardo de $T_d = 200$ ms (e.g., operación transcontinental), ¿qué modificaciones al controlador serían necesarias? Mencionen al menos una técnica del estado del arte (e.g., wave variables, model mediation).

(d) **Aplicación real:** Propongan una aplicación concreta en la industria o medicina donde su sistema, con las mejoras indicadas, podría tener impacto. Estimen el orden de magnitud del costo de implementación.

**(a) Limitaciones del sistema implementado.**

I. *Precisión del estimador de fuerza ($\epsilon \approx$ 1–3 N):* el modelo gravitacional usa masas y centros de masa del YAML de UFACTORY sin incluir fricción articular. Esto introduce un error residual que limita la sensibilidad a contactos $<$ 2 N. *Solución:* usar el sensor F/T nativo del xArm Lite 6 mediante el tópico `/{ns}/uf_ftsensor_ext_states` de `xarm_ros2` (ya disponible en el repositorio [2]), o calibrar el modelo dinámico completo con regresión de parámetros dinámicos [3].

II. *Latencia DDS + Python ($T_{total} \approx$ 5–20 ms):* la serialización CDR de ROS 2 y el GIL de Python limitan la frecuencia de control efectiva. *Solución:* implementar los nodos de admitancia/impedancia en C++ con rclcpp y ejecutar con política SCHED_FIFO para tiempo real duro en Linux.

III. *Control traslacional sin orientación del efector:* el sistema comanda únicamente los 3 DOF lineales del EE mediante `TwistStamped.linear`, ignorando la orientación. En tareas de ensamble o cirugía, la orientación del efector es crítica. *Solución:* extender el `TwistStamped` con las componentes angulares y aumentar el modelo de admitancia a 6 DOF, lo que MoveIt Servo soporta de forma nativa.

**(b) Comparación con el estado del arte.** Niemeyer et al. [8] reportan sistemas de teleoperación bilateral con retardos de hasta 500 ms usando variables de onda, con errores de fuerza del orden de 0.5 N usando sensores F/T dedicados. Nuestro sistema, sin sensor F/T externo, obtiene errores de $\approx$1.5 N, lo que es competitivo para aplicaciones de baja precisión. En latencia, nuestro sistema en LAN ($T_d < 2$ ms) supera en rendimiento a la mayoría de implementaciones inalámbricas reportadas, aunque el uso de Python introduce jitter de software no presente en implementaciones en tiempo real duro (RTOS).

**(c) Escalabilidad para $T_d = 200$ ms.** Con $T_d = 200$ ms y $\omega_c = 314$ rad/s, el criterio de Colgate da $K_{d,max} <$ 0.025 N/m, prácticamente nulo. La solución estándar para

este régimen es el **método de variables de onda** (*wave variables*) [8]: transformar las señales de fuerza y velocidad en variables de onda que preservan la pasividad independientemente del retardo:

$$u = \frac{1}{\sqrt{2b}}(b\,v + f)$$
$$v^* = \frac{1}{\sqrt{2b}}(b\,v - f)$$

donde $b$ es la impedancia característica del canal. Otra técnica es *model mediation* [8], que usa un modelo local del entorno en el maestro para generar háptica local mientras el esclavo se sincroniza de forma asíncrona.

**(d) Aplicación propuesta: cirugía laparoscópica remota.** El sistema, mejorado con sensor F/T de muñeca, control de pose completo y variables de onda, podría aplicarse en cirugía laparoscópica teleasistida para hospitales rurales con conexión a especialistas metropolitanos. El costo estimado de implementación sería del orden de $15,000–$30,000 USD (dos xArm Lite 6 + sensores F/T + infraestructura de red de baja latencia), frente a sistemas comerciales como da Vinci que superan $2 M USD.

## IX. Conclusiones

*Pregunta Cn1 [6 pts]:* Redacten una sección de conclusiones que responda:

(a) ¿Lograron implementar teleoperación con sensado de fuerza funcional entre los dos xArm Lite 6? ¿Con qué nivel de desempeño (cuantifiquen)?
(b) ¿Cuál fue el mayor desafío técnico encontrado y cómo lo resolvieron?
(c) ¿Qué aprendizaje clave obtuvieron sobre la relación entre el Jacobiano, los torques articulares y la fuerza cartesiana?
(d) ¿Qué harían diferente si repitieran el proyecto?

Se implementó exitosamente un sistema de teleoperación bilateral con sensado de fuerza entre dos manipuladores xArm Lite 6, basado en ROS 2 Humble y MoveIt Servo, operando a 50 Hz mediante comunicación DDS sobre red LAN. El controlador de admitancia en el maestro ($\boldsymbol{M}_d = \mathrm{diag}(2,2,2)$ kg, $\boldsymbol{B}_d = \mathrm{diag}(10,10,10)$ N·s/m) y el seguidor de impedancia en el esclavo ($\boldsymbol{K}_p = \mathrm{diag}(5,5,5)$ s$^{-1}$) producen teleoperación cartesiana fluida con protección automática ante singularidades ($\kappa > 50$). El estimador de fuerza basado en residuos de torque articular demostró ser funcional para detectar contactos con objetos rígidos, con umbral de detección calibrado automáticamente de $F_{th} \approx 1.5$ N y error de estimación de $\approx$1.5 N frente a referencia con báscula. Dos sensores ESP32 con micro-ROS proveen fuerzas de entrada del operador y del esclavo independientes del modelo dinámico, con parámetros diferenciados ($k = 1000$ vs. 1500 N/m, rango $\pm 3$ vs. $\pm 6$ N) para adaptarse a los distintos rangos de fuerza en cada rol.

El mayor desafío técnico fue la integración de MoveIt Servo con dos namespaces simultáneos y la compatibilidad QoS entre micro-ROS (`BEST_EFFORT`) y los nodos ROS 2 (`RELIABLE`). La solución consistió en remaear los tópicos TF por namespace y configurar explícitamente los perfiles QoS en los suscriptores de los sensores físicos. El soporte para Isaac Sim permitió validar la arquitectura de teleoperación sin robots físicos, reduciendo el tiempo de integración.

El aprendizaje fundamental de este proyecto fue la comprensión práctica de la dualidad $\boldsymbol{\tau} = \boldsymbol{J}^\top \boldsymbol{F}$: la misma relación que permite estimar fuerza a partir de torques (dirección análisis) fundamenta el modelo de admitancia que convierte fuerza de contacto en velocidad cartesiana del maestro (dirección síntesis). Esta dualidad es el corazón de la teleoperación bilateral y queda explícita en cada ciclo de control del sistema implementado.

Si se repitiera el proyecto, la primera mejora sería extender el control a los 6 DOF cartesianos del efector (posición + orientación), habilitando tareas de ensamble de precisión. En segundo lugar, se implementaría el lazo de estimación de fuerza en C++ con tiempo real duro, reduciendo la latencia del estimador de $\approx$ 1 ms a $<$ 0.1 ms. Finalmente, se explotaría el sensor F/T nativo del xArm Lite 6 (tópico `uf_ftsensor_ext_states` de `xarm_ros2`) para obtener referencia de fuerza de mayor precisión sin hardware externo adicional.

## Agradecimientos

Los autores agradecen al Computational Robotics Lab del Tecnológico de Monterrey y al Prof. Alberto Muñoz por el acceso a los robots xArm Lite 6 y por la orientación técnica durante el desarrollo del proyecto. Se agradece también al equipo de UFACTORY por la documentación abierta del SDK y los parámetros cinemáticos del xArm Lite 6 [1].

## Referencias

[1] UFACTORY, "xArm-Python-SDK," GitHub repository, 2024. [Online]. Available: https://github.com/xArm-developer/xArm-Python-SDK
[2] UFACTORY, "xarm_ros2," GitHub repository, commit 5bb832f, 2024. [Online]. Available: https://github.com/xArm-Developer/xarm_ros2
[3] B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, Springer, 2009.
[4] M.W. Spong, S. Hutchinson, M. Vidyasagar, *Robot Modeling and Control*, Wiley, 2006.
[5] N. Hogan, "Impedance Control: An Approach to Manipulation," *ASME J. Dyn. Sys., Meas., Control*, vol. 107, 1985.
[6] J.E. Colgate, J.M. Brown, "Factors affecting the Z-width of a haptic display," in *Proc. IEEE ICRA*, 1994, pp. 3205–3210.
[7] R.M. Murray, Z. Li, S.S. Sastry, *A Mathematical Introduction to Robotic Manipulation*, CRC Press, 1994.
[8] G. Niemeyer, C. Preusche, G. Hirzinger, "Telerobotics," in *Springer Handbook of Robotics*, Springer, 2008.
[9] S. Thompson, I. Reid, **A. Muñoz**, D.W. Murray, "Using a stationary camera to improve the performance of a teleoperated robot," *IEEE Trans. Systems, Man & Cybernetics-A*, vol. 31(1), pp. 74–82, 2001.
[10] A. Autor, B. Autor, "Título del artículo," *IEEE Trans. Robotics*, vol. XX, no. Y, pp. ZZ–ZZ, 202X.
[11] C. Autor, D. Autor, "Título del artículo," in *Proc. IEEE ICRA*, 202X, pp. ZZ–ZZ.

## Apéndice

| Pregunta / Criterio | Pts. | Mín. | Evidencia Requerida |
|---|---|---|---|
| **Teoría y Fundamentos** | | | |
| T1: Jacobiano y singularidades xArm | 8 | 5 | Derivación + figura sing. |
| T2: Control de par calculado (CTC) | 10 | 6 | Derivación + gráfica resp. escalón |
| T3: Control de impedancia | 8 | 5 | Cálculo $K_{d,max}$ + tabla $K_d$ |
| **Arquitectura y Comunicación** | | | |
| A1: Diagrama de bloques completo | 10 | 6 | Figura con señales dimensionadas |
| A2: Protocolo y métricas de red | 8 | 5 | Tabla RTT/jitter + captura ping |
| **Sensado de Fuerza** | | | |
| F1: Estimador torque-fuerza | 12 | 7 | Gráficas antes/después filtro |
| F2: Comparativa estrategias | 8 | 5 | Tabla completada + justificación |
| **Control e Implementación** | | | |
| C1: Control del esclavo | 10 | 6 | Código funcional + flowchart |
| C2: Feedback háptico en maestro | 8 | 5 | Cálculo $\beta_{max}$ + código |
| I1: Setup y configuración xArm SDK | 10 | 6 | Documentación reproducible |
| **Resultados Experimentales** | | | |
| E1: Setup y descripción experimentos | 6 | 4 | Tabla hardware + descripción |
| E2: Error de seguimiento (6 joints) | 12 | 7 | Gráficas + tabla métricas |
| E3: Estimación y validación fuerza | 14 | 9 | Gráficas + validación báscula |
| E4: Análisis estabilidad háptica | 8 | 5 | Prueba hard-wall + $\beta_{crit}$ |
| **Análisis y Conclusiones** | | | |
| D1: Discusión limitaciones y EA | 8 | 5 | Citas IEEE + comparativa cuant. |
| Cn1: Conclusiones | 6 | 4 | Párrafos 200–300 palabras |
| **Calidad del Documento IEEE** | | | |
| Formato y ortografía | 4 | 2 | LaTeX compilable, sin errores |
| Figuras con pie de figura y etiquetas | 4 | 2 | Mín. 8 figuras propias |
| Referencias (mín. 10, 2 recientes) | 4 | 2 | BibTeX correcto |
| **TOTAL** | **160** | **96** | |

- Score $\geq$ 96 puntos sobre 160 (60 %).
- La sección E3 debe incluir **evidencia experimental real** con los xArm físicos (no simulación).
- El estimador de fuerza debe mostrar coherencia con los torques articulares medidos: $\boldsymbol{\tau}_{meas} \approx \boldsymbol{J}^\top \hat{\boldsymbol{F}}_{ext}$.
- El documento LaTeX debe compilar sin errores.
- Mínimo **8 figuras originales** (no tomadas de internet).

### ▷ Checklist de entrega

Antes de entregar, verifiquen que el repositorio/zip contiene:

- ☐ `reporte.pdf` — artículo IEEE compilado (este documento completo)
- ☐ `reporte.tex` — fuente LaTeX del artículo
- ☐ `master_control.py` — código del robot Maestro
- ☐ `slave_control.py` — código del robot Esclavo
- ☐ `force_estimator.py` — módulo de estimación de fuerza
- ☐ `net_test.py` — herramienta de test de red (adaptado para xArm)
- ☐ `data/` — carpeta con los archivos `.csv` o `.npy` de cada ensayo experimental (mínimo 3 ensayos)
- ☐ `figures/` — imágenes originales (PNG/PDF) usadas en el reporte
- ☐ `video_demo.mp4` — video de 60–120 s mostrando el sistema funcionando en tiempo real con ambos xArm
- ☐ `README.md` — instrucciones para reproducir los experimentos

Un video de demostración de alta calidad que muestre:

- El operador moviendo el Maestro y el Esclavo replicando el movimiento.
- Un evento de contacto claro con las gráficas de fuerza en pantalla.
- La reacción háptica perceptible en el Maestro.

Un video bien editado (subtítulos, señales anotadas) puede obtener hasta **+10 puntos extra** a criterio del profesor.